ME/CS/EE 134 Final Report

Riti Agarwal, Iñigo Jensen, Kimberly Miranda Reyes

19 March 2025

1 Introduction

Our robot's goal is to take turns playing a game of mini-golf with another player. It can pick up a club, hit the ball toward the hole, and reposition it away from the walls or toward the starting position. Due to the nature of the game, a core challenge is ensuring the ability to swing a club in the correct orientation at every location on the designed golf course. Needing to be a dexterous robot with a fixed base, this becomes a non-trivial task. Additionally, the playing area on the mini-golf course is encapsulated by walls, creating obstacles that the robot must avoid during its movements and tasks.

2 Mechanisms and Hardware

2.1 Overall design

Due to the reachability requirements natural to a game of mini-golf, holding the motors needed to manipulate the club at long distances across the course becomes a mechanical limitation. Unless restricted to a small course, the torque required (>16 N) would exceed the continuous torque limits of the available motors. Thus, a SCARA (Selective Compliance Articulated Robot Arm) layout was chosen for the first two links. This layout places the axis of rotation of our first three motors parallel to the direction of gravity (i.e. the links stay parallel to the ground throughout their range of motion). Additionally, the first/base motor is mounted at a location above the course. The final two arm motors are mounted parallel to the third motor, facing the course below. A gripper is attached from the fifth motor and a bike cable runs from the gripper, along the arm, to a motor mounted above the base motor.

2.2 DOFs

Defining the 2D area of the golf course as xy-space and z as the vertical direction (opposite gravity), the first two motors/DOFs (degree of freedom) in the SCARA layout allow the robot to reach any (x,y) position of the course (provided the necessary link lengths). The third motor/DOF, at the end of the second link, allows the robot to orient itself in the required swing direction; we define this angle in xy-space as θ. To decide the final DOFs needed to manipulate a club to hit the golf ball, the shape of a mini-golf swing was considered; a mini-golf swing can be conducted entirely in a plane that spans a vector in z and in θ. Thus, only two motors/DOFs are required after the third motor. These two motors are used to reach the desired z position at contact (or other task) and the angle within the swing plane ($span\{\hat{z}, \hat{\theta}\}$). We define ϕ as the angle within this swing plane. The third link holds the fourth motor with its axis of rotation at 90° from the axis of rotation of the

third motor. The fifth and final motor is planar with the fourth. This brings the DOFs of the arm to a total of 5. Finally, to grab the club or ball, a gripper is needed, attached to the fifth motor. The overall layout can be seen in Figure 1.

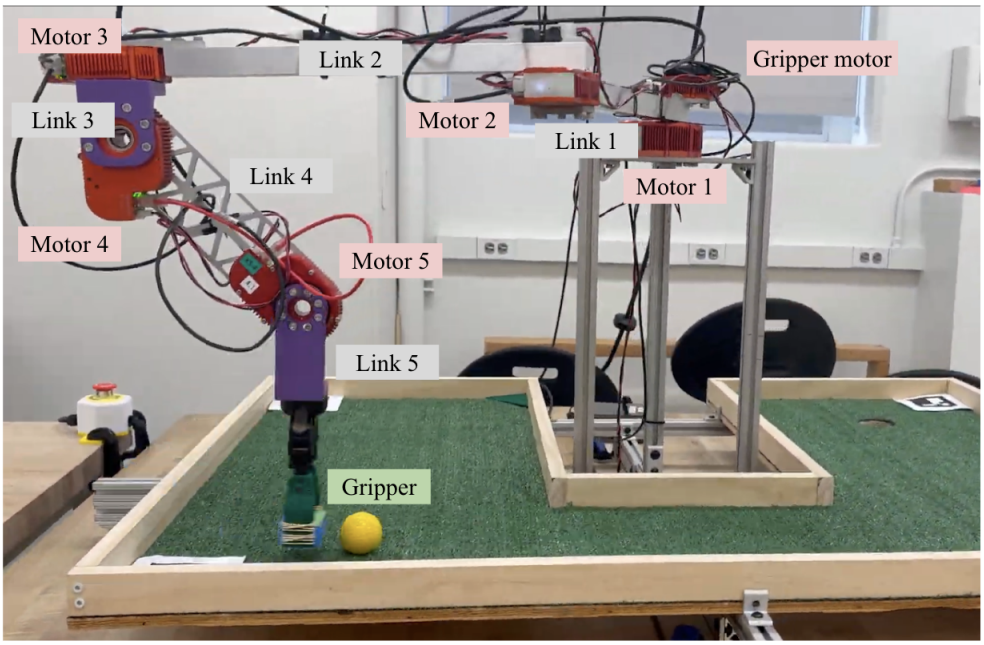


Figure 1: Mechanical layout of the robot.

2.3 Hardware

2.3.1 Course

Figure 1 shows the "C-shaped" layout of the mini-golf course, wrapping around the base motor stand. This layout was chosen for both aesthetic and design reasons. It makes achieving a hole-in-one more challenging than an open shot on a rectangular or similarly shaped course, oversimplifying the task. Also, the open section of the C provides a space for cabling. Due to the SCARA layout, the greatest constraining factor for the course size became the camera field of view (FOV) and table space. Mounted on the ceiling and looking down, the camera could see just under 5 ft x 4 ft of table space. Therefore, considering the conical FOV, a 4 ft x 3 ft course was selected. Standard 1 in x 2 in wood planks were used for the walls (nominal size of 0.75 in x 1.5 in). These walls are just shorter than a standard golf ball (1.68 in diameter), allowing the golf balls to bounce off and minimizing the obstacle size the robot needs to avoid during manipulation.

2.3.2 Links

The 4 ft x 3 ft course required a combined first and second link length of ∼0.8 m. For the links, three main options were available: balsa wood, PLA, or aluminum. Due to the long-moment arm, aluminum was chosen as the material. Two options were available for the aluminum material: 1 in x 1 in square tubing or 1/4 in plates. Mounting to the stator and rotor of a HEBI motor is done through a bolt pattern on either of the large faces. The axis of rotation of the HEBI motor is normal to the bolt pattern planes, thus mounting the 1/4 in plate would align the weak bending axis

(smaller cross-sectional moment of inertia) with the force of gravity. Instead, the square tubing has a cross-sectional moment of inertia similar to an I-beam, which is very rigid in bending. Thus, the square tubing was selected for the first two links. After two iterations of adapters, we count mount the square tubing directly to the rotor side of the base and second motors with four bolts. The first iteration was our long "spoon" (left of Figure 2) which was needed for the bolt heads connecting the adapter to the tube to clear the motor. A second iteration used a ring spacer to eliminate this need for clearance (middle of Figure 2). The bolt pattern on the stator side however was too large to bolt directly to the square tubing, thus an aluminum plate adapter was used ("fork" pictured on right side of Figure 2).

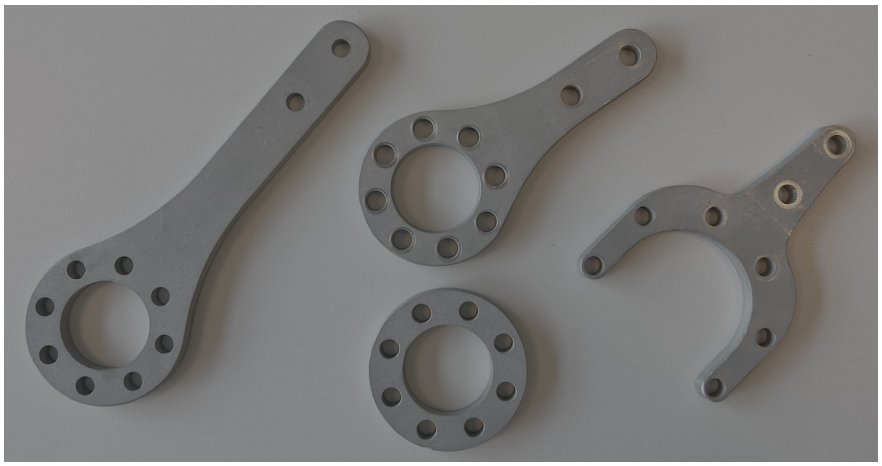


Figure 2: Adapters for mounting tubes to motors.

The third link needed to hold the fourth motor 90° rotated from the third motor: an "L-bracket". Needing a complicated form and not needing length, we opted to 3D print this L-bracket. The same reasoning applies for the L-bracket mounting the gripper to the fifth motor. The fourth link between the fourth and fifth motors raises and lowers the gripper and puts a plate mounted to the motors in bending along its strong axis. Thus, an aluminum plate was used for this link. A triangular truss structure was pocketed into the link for weight reduction as it is held at a long moment arm from the base. This link can be seen in Figure 1.

2.3.3 Gripper & club

Grabbing the ball and club with the same gripper presented a unique challenge. The initial version of the club had a shaft of ∼5 in. The robot was to grab the club as it laid on the course, pick it up, rotate to vertical, and then swing. For this manipulation, attachments with long cutouts matching the shaft diameter and a cutout for the ball were designed and printed. Just before pivoting to a shaft-less club, a knob for the club shaft was printed so the robot could release the club slightly, allow it to drop to the knob, and re-grip at the same position every time.

The first iteration for the shaft-less club and gripper attachments featured a pill-shaped extrusion on the attachments and a cut into the sides of the club. This shape ensured the club was fully constrained with the grippers closed while also being symmetrical when the club was flipped. This pill shape had a radius of 2.5 mm which gave roughly a tolerance of 2.5 mm radius in the $z\theta$ plane. The second iteration featured a rectangular prism shape with a larger tolerance. While it provided a larger tolerance, the shallow angles of the shape and nearly square base allowed for some rotation of the club in the ϕ angle, even while the grippers were clasped with high torque. Even with added

tolerance, we struggled with accuracy, so a final version of the gripper attachments was printed with a wider grasp and flat side on which rubber bands were wrapped around for grip. A final club with a smaller width and flat sides was printed for increased tolerance.

2.3.4 Framing, computing, and power

Aluminum extrusions were used to constrain the course and these extrusions were mounted to the base motor stand. The entire structure was clamped to the table. The visual sensing capabilities of the robot come entirely from an RGB camera mounted to the ceiling framing, directly above the base motor. The HEBI motors connect in series for communication; each motor receives one Ethernet cable with communication and sends communication through another Ethernet cable to the following motor/computer. A power distribution box runs in series with the power to all motors along with a quick-release kill button, to shut off power to the motors if necessary. An Intel NUC is used as the computer for the robot.

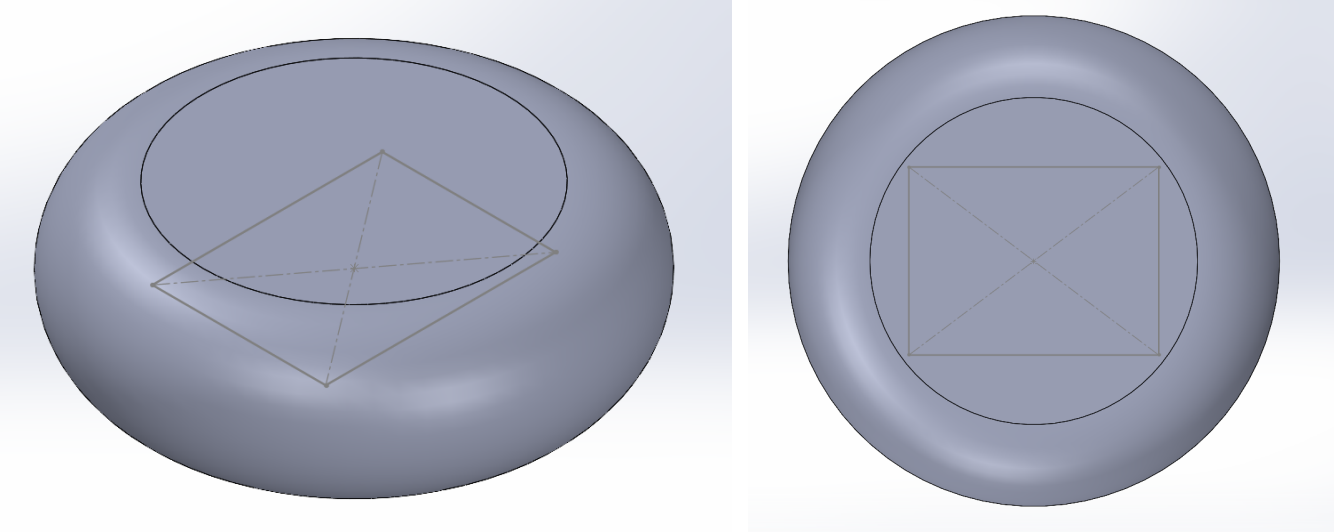
2.4 Motor selection

Motors from the HEBI X5 and X8 series were available for use. To determine the right motor for each joint, it was necessary to determine the minimum torque requirements. Due to the SCARA layout of the first three motors, the torque due to gravity acting on these motors is in an axis perpendicular to their axis of rotation. Thus, the torque acts on the bearings between the rotor and stators of each motor and does not require electrical power. According to HEBI hardware documentation, the maximum static cross-moment torque the X5 and X8 motors can sustain is 55.6 Nm. This is more than enough to hold 5 motors of 330-480 g at 1 m. The arm motor with the most torque needed to fight gravity is the fourth motor. The base motor also experiences relatively higher torque during acceleration due to the high rotational momentum of the extended arm in addition to any internal friction which is likely increased by the cross-moment load. Thus, the strongest X8-16 motor with up to 16 Nm of continuous torque was used as the base motor, and the X8-9 with up to 8 Nm of continuous torque was used as the fourth motor. The X8 motors have more power than the X5 and the second number correlates to the maximum continuous torque. Thus an X8-9 has the same maximum torque as an X5-9, but also a higher top speed due to the additional power. X5-4 motors were used for the rest of the joints and for the gripper. The gripper didn't need more than 2 Nm of torque to grip the club or the ball.

3 Basic Robotics

3.1 Task space

If the robot were run in simulation and the robot was allowed to fold into itself, the idealized task space would appear as in Figure 3. In real life, however, the stand for the base motor, the golf course, and the robot itself create obstacles that reduce the actual task space. The base motor stand creates a roughly triangular prism-shaped hole in the middle of the idealized task space. The course is set at a height that is just above the lower surface of the task space (i.e. the gripper can touch the course a little bit before full extension of motors 4 and 5). This allows the robot to pick up objects anywhere on the C-shaped course and maximizes the task space we have in the z-direction. The range of the fourth motor was limited so that the robot would not swing up and hit itself. This roughly removes the top half of the task space. A final limit to the task space is the limits that were set on the base motor so that the robot stays away from the cables that run in the "cutout" section of the course. This takes a slice off of the back of the task space.



(a) Task space isometric view.

(b) Task space top view.

Figure 3: Idealized task space with course outline.

3.2 Kinematics

3.2.1 Forward kinematics

The task coordinates as described in the mechanical section include x, y, z, θ, and ϕ. We can represent the task coordinates and joint coordinates as vectors:

$$\vec{x} = \begin{bmatrix} x \\ y \\ z \\ \theta \\ \phi \end{bmatrix}, \quad \vec{q} = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ q_5 \end{bmatrix}$$

The coplanar nature of the first three motors reduces the complexity of the forward kinematics, allowing us to write the task coordinates as a function of joint coordinates as follows:

$$\vec{x} = \begin{bmatrix} x \\ y \\ z \\ \theta \\ \phi \end{bmatrix} = \begin{bmatrix} l_1(\sin(q_1) + \sin(q_1 + q_2)) + \cos(\theta) \cdot l_4(\sin(q_4) + l_3 \sin(\phi)) \\ l_1(\cos(q_1) + \cos(q_1 + q_2)) + \sin(\theta) \cdot l_4(\sin(q_4) + l_3 \sin(\phi)) \\ h_{M4} - l_4 \cos(q_4) - l_5 \cos(\phi) \\ q_1 + q_2 - q_3 \\ q_4 - q_5 \end{bmatrix}$$

From the forward kinematics, we can calculate Jacobian matrix which allows us to translate between task space and joint space. A helper function (KinematicChain) which takes measurements from the robot URDF was used to calculate the task coordinates as xyz position and a rotation matrix. It also outputs the xyz Jacobian matrix rows (J_v) and the cartesian rotation matrix angle rows of the Jacobian matrix (J_ω) for a given joint coordinate.

3.2.2 5 DOF inverse kinematics

Our Newton-Raphson (NR) solver for the 5DOF arm can either use a provided init_guess or automatically create one based on the target's Cartesian position. This is important because the robot

can reach the same point in space through 8 different joint configurations (branches). Controlling which branch the solver picks was crucial to avoid crashing into walls and to stop the base motor from rotating in ways that are slow or risky.

To help control this, we assign an initial guess based on which side of the workspace the target is on. For example, if the target is on the robot's right side ($x > 0$), we set the base motor (joint1) to $+\pi/3$. If the target is on the left ($x < 0$), we set it to $-\pi/3$. This helps the robot start with the base joint already turned toward the goal, so it doesn't need to spin around too far, which could cause it to hit cables or workspace boundaries.

Since NR is a local optimizer, starting with a good guess helps it stay close to the desired branch. We also found that if we started near a singularity—a position where the robot's joints are stretched out or folded—the solver would sometimes switch to a different branch right away, leading to the wrong motion. To prevent this, we made sure our guesses avoided singularities.

This was most important when lowering the arm. For example, when picking up the club or ball from above, changing to a different base joint branch could cause the base to rotate while the arm is lowering, making it hit walls or other obstacles. To avoid this, we also used the current joint positions (like the shoulder, elbow, and wrist angles) to help guide the solver and keep it near a safe, expected configuration.

When we needed to move just a short distance or when it was critical to stay on the same branch—such as when moving the ball away from walls after picking it up—we skipped the quadrant-based guess. Instead, we used the robot's current joint position as the initial guess to make sure we didn't accidentally switch branches.

However, when doing longer moves—such as taking the ball to the start or moving toward a new hitting position—it was fine to change branches, so we went back to using the quadrant-based initial guess.

3.2.3 3 DOF inverse kinematics

In order to avoid the walls of the golf course, a second inverse kinematics function was written which locked the final two motors and translated to a desired x, y, and θ. This way we can raise the arm to a position above the walls and call this inverse kinematics function to move the arm without the risk of hitting the wall. If the full 5 DOF function is used, the arm switches branches, moving the final two motors, and frequently hitting the course during translation. To do this, only the first two rows and first three columns of J_v are used. This gives us the x and y coordinates in terms of the first three motors. Finally, to get θ in terms of joint angles we can look at the forward kinematics equation. This gives us a simple third row for the complete Jacobian: $\begin{bmatrix}1 & 1 & -1\end{bmatrix}$. From this we compute our Newton-Raphson algorithm in a similar manner to the 5 DOF case. The difference is we set $\dot{q}_4 = 0$ and $\dot{q}_5 = 0$ to keep the final two motors locked.

For the initial guess for this ikin, we use a similar quadrant-based approach as the NR solver above. But, since this 3 DOF ikin has fewer constraints than the full 5DOF NR solver (it only cares about x, y, and θ, not z or full orientation), giving it an initial guess for all 5 motors definitely over-constrained the solver, and so sometimes it wouldn't converge in our given iterations (we even tried increasing iterations, and realized that some cartesian positions are impossible to converge to when you give it 5 joint angles instead of 3). So, if the 3 DOF ikin doesn't converge the first time, we retry, this time giving it an initial guess for only the 3 motors that control x, y and theta. By doing this, we ensure that the 3 DOF ikin always converges, and we have better control of which branch we want the robot to be at.

3.2.4 Iterative Newton-Raphson

As we struggled to get the two Newton-Raphson functions to give final joint coordinates within the same branch as the initial joint coordinates (even for very small task coordinate translations of a couple cm), we attempted to iteratively run Newton-Raphson algorithms, slowly moving the final desired task position linearly from the overall start to finish. This theoretically would help limit large jumps that could potentially cause jumps to other branches while running the original Newton-Raphson algorithms. We found limited success with this method and decided to pivot due to increased computation. It was determined that either the initial joint positions that we were starting at were near singularities or right at the motor limits causing near 360° jumps for a motor.

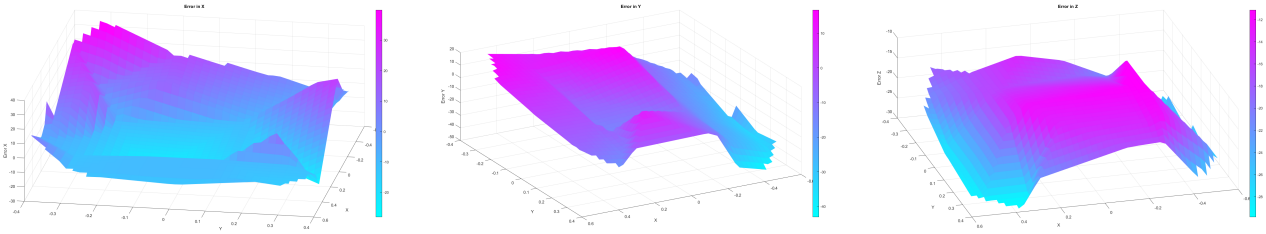
3.3 Trajectories & continuity

The HEBI motors used have position, velocity, and torque control algorithms built in. Thus, we pre-calculated trajectories assuming perfect tracking and real-time computing at 100 Hz, and allowed the built-in motor controls to handle the errors with built-in feedback loops. In order to ensure continuity, quintic splines in joint coordinates were used for the trajectories. By default, the starting and ending joint velocities were set to 0 rad/s. Early on, when collisions were detected via high torque on a motor, the trajectory would be replaced and the current initial velocities needed to matched by the initial accelerations of the new trajectories in order to ensure continuity of the acceleration. It was determined that returning to home position if a collision was detected was unsafe, and so this was replaced by halting commands to the motors.

3.4 Precision

3.4.1 Error mapping

Despite attempts to increase the rigidity of the first two links, there continued to be slop which caused large errors in (x,y,z). The largest source of this slop was the "fork" adapter holding the second motor to the first link. As the second joint angle, q_2 rotated away from 0 and to a magnitude of $\pi/2$, the moment arm acting torsionally on the "fork" increased. This caused various errors based on q_2. This means that even for the same (x,y) position, different θ, ϕ, and branches gave very different errors in (x,y,z). It was considered that an error map would be created to account for multiple branches at every (x,y) position on the course. Due to the lack of time, we opted to map from (x,y) to (x,y,x) error. The error at 61 points around the course was measured. A large focus was dedicated to points along the wall as this was where the ball ended up a lot of the time. Plots of the x, y, and z error can be seen in Figure 4.



(a) Error in x direction vs (x,y). (b) Error in y direction vs (x,y). (c) Error in z direction vs (x,y).

Figure 4: Error maps in (x,y).

3.4.2 Haptic grip detection

In order to verify that the ball or the club was picked up, we initially check to see if the item was still found in the same location. It also had to move out of the way to double-check that it wasn't just covering the item. Since the gripper was controlled by a HEBI motor with live torque readings, it was realized this could be used for haptic feedback. The gripper closed positions were determined by finding the position at which -2 Nm of torque was experienced by the gripper motor for each the ball and the club. If the grippers were to miss the item when attempting to pick it up, we would see less torque. Thus, instead of checking for visual confirmation that an item was picked up, the gripper motor torque was used instead. A threshold was found between torque holding and not holding the items.

3.4.3 Adding sag to URDF

Upon first analysis of the sag in the links, it was thought that there was a consistent $\sim 2.5^\circ$ of sag per link for the first two links. Thus, it was considered that a 2.5° sag should be added to the URDF of the robot. Had this been the case, this would have likely reduced the need for an error map in order to precisely pick up and manipulate items. Upon further analysis, however, most of the sag came as described in the error map section: from the "fork" attaching the second motor to the first link, which experienced varying torques and thus sag based on the angle of q_2. Therefore, we would have still experienced varying error between our modeled robot and real robot.

4 Cameras and Visual Detectors

4.1 Camera setup and calibration

We use a single RGB camera running at 15 Hz to detect all our mini-golf-related objects. The detector node receives the frames from the camera, identifies the objects, and precisely maps each object's location to world coordinates. The system is responsible for identifying the robot's golf ball (a yellow colored ball), the user's golf ball (a light green ball), the golf club (represented by a green strip next to a blue semicircle), and a blue semicircle marker, which serves as a spatial reference point within the environment.

Prior to running our detector node, we did a one-time camera calibration using the chessboard calibration method to estimate the camera's intrinsic matrix and distortion coefficients. This calibration was done after physically mounting the camera to ensure correct extrinsic with respect to the workspace. We then use these intrinsic parameters for distortion correction in our detector pipeline.

We also set our camera parameters to ensure consistent image quality across varying lighting conditions: brightness set to 128, contrast to 128, saturation to 160, white balance fixed at 4000 (with auto white balance disabled), and exposure set to 250 with auto-exposure turned off.

4.2 Object detection pipeline

Once the camera streams images, each frame is converted from RGB to HSV color space to allow for color segmentation using pre-calibrated HSV thresholds. Using the provided code for HSV tune, we manually tuned HSV values for each object type: the green strip, blue marker, and both types of balls. Morphological operations, such as erosion and dilation, follow to reduce image noise and improve the quality of the object masks. This is performed on all masks except for the green strip (when we tried it for the green strip the strip couldn't be detected anymore, even when we

dilated first). Afterwards, the system detects contours using OpenCV's contour-finding algorithm (cv2.findContours()). For the golf balls, we perform a circularity check by fitting a minimum enclosing circle and calculating the overlap ratio between the contour and the circle, accepting contours as circles only if this ratio exceeds 0.6 (we printed the overlap ratio and it typically never went below 0.75, and since we never had any other light green or yellow colored objects we were able to lower the threshold a little bit). The green strip is detected by fitting a minimum area rectangle to the largest green contour, which is then used to determine the club's orientation (angle of the green rectangle). The position of the golf club was determined to be halfway between the center of the blue contour and the center of the green contour.

4.3 World coordinate mapping

To localize objects in world space, we use the homography computed from an ArUco marker grid. Four ArUco markers (IDs 1 through 4) are placed at the edges of our golf course in a rectangular arrangement, each with known world coordinates relative to the grid center. For each frame, the system detects visible markers and uses their image coordinates and corresponding world positions to compute a perspective transform matrix. Prior to this step, pixel coordinates of both the markers and detected objects are undistorted using the camera's intrinsic parameters.

The system is designed to deal with one ArUco marker being covered. If only three out of four ArUco markers are visible, the system estimates the position of the missing marker using the known rectangular grid layout. Specifically, it reconstructs the missing marker's position using the diagonal markers and their geometric relationship with the fourth detected marker.

Next, the detected object's pixel coordinates are mapped to real-world coordinates, where we were able to get the detector accurate to a millimeter.

4.4 System integration and ROS publishing

The system then publishes this information as ROS messages containing object positions and, in the case of the green strip, orientation quaternions derived from the green rectangle's principal axis.

Since we are able to detect our mini-golf-related objects accurately enough in every frame, we felt our detector was able to deal with every situation we needed to handle in our brain. It could deal with 1 ArUco marker being covered, which would happen when the robot was moving around, and it could deal with changes in lighting. We dealt with our robot covering the objects in our brain, and so our detector was able to cover all expected situations.

5 ROS Software Architecture

In the diagram above, the publisher `detected_objects` contains the position and orientation of all the mini-golf related objects that are essential to the robot's turn. So, this includes the golf club and the robot's golf ball. Next, the `user_ball` message contains the position of the user's ball. The `spline_segments` publisher contains all the spline segments the brain calculates and enqueues for the lowlevel to execute. Additionally, the `control_commands` are what we defined in case the brain needs to send halt or replace commands to the lowlevel (in case of collision). The lowlevel also publishes the `execution_status`, which tells the brain whether or not the lowlevel has completed all the splines, or if there was a collision that was detected. Finally, the lowlevel publishes the position and velocity to the motors using `joint_commands`. The motor publishes the actual position (joint coordinates), velocity and torque the motors are at, and we use this in our lowlevel and brain for various tasks.

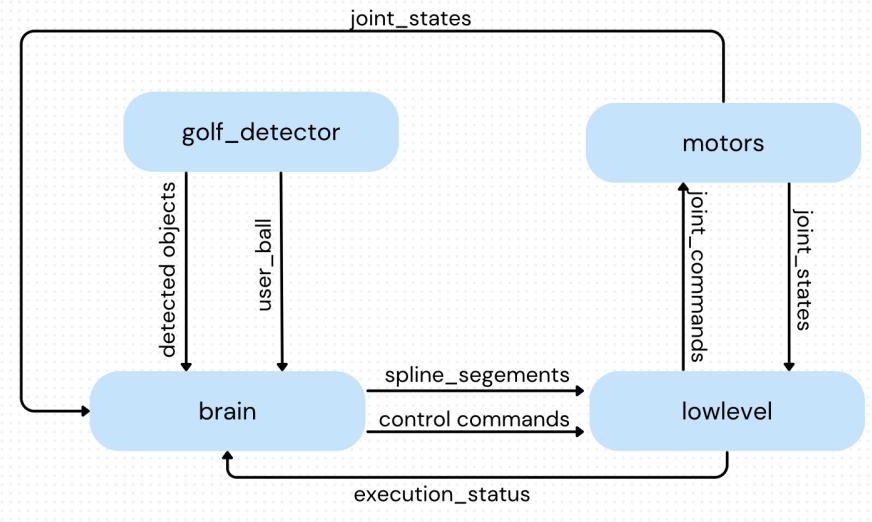


Figure 5: ROS high level architecture

5.1 ROS nodes

We have 3 main ROS nodes:

1. **golf_detector:** The main function that is triggered here is the `process` function, which finds the position and orientation of all the mini-golf related objects, converts it to world coordinates, and publishes this information for the brain node to use. The `process` function is triggered every time a new frame comes in - so this runs at the camera's frame rate, which in our case is 15 Hz.

2. **Brain:** This controls all the high-level logic, planning, and deciding what the robot does (pick up club, move ball, hit ball, etc.). It takes in the information from the golf detector and lowlevel (whether lowlevel is ready to take new commands or if a collision has been detected) and makes decisions on what to do. It then publishes joint coordinates based on the necessary actions to the lowlevel to execute. In our brain, we have 4 callback functions:

   (a) `recv_objects` - this is the main control loop where high-level decision-making happens. It decides whether to move the ball to the starting position, pick up the club, move ball away from wall, hit or putt the ball depending on position or switch to user turn when needed. This callback is triggered every time a message is received on the `detected_objects` subscriber, but will not be triggered if `self.ready` is false (if the robot shouldn't move or lowlevel isn't ready).

   (b) `recv_status:` This callback listens to the `/execution_status` topic published by the LowlevelNode. It updates the `self.ready` flag depending on whether the LowlevelNode reports it is ready (i.e., no more splines to execute) or if a collision has occurred (if collision occurred it asks the robot to stop moving - so that it doesn't hit anyone).

   (c) `recv_user_ball:` This function monitors the position of the user's ball (published as an `Object` message on `/golfdetector/user_ball`). It determines when the user's ball has stopped moving after having moved, concluding the user's turn and allowing the robot's to resume.

(d) `recv_fbk:` This function listens to joint feedback on the `/joint_states` topic. It tracks joint positions, velocities, and efforts, which are used in kinematics calculations and logic for tasks such as determining if the club has been successfully gripped.

3. **lowlevel:** The LowlevelNode is responsible for executing joint-level commands on the robot. It subscribes to spline commands from the BrainNode and translates them into position and velocity commands for the motors at the current time, which are sent to the robot's actuators. Additionally, it monitors joint efforts to detect unexpected collisions during execution. It has 4 callback functions:

   (a) `recvspline` - Triggered when a new `SplineSegment` is received from the `/spline_segment` topic. This function adds the segment to a queue of trajectories for execution.

   (b) `recv_control` - Triggered when a `ControlCommand` is received on the `/control_command` topic. It handles special commands like "halt" (stop immediately) and "replace" (clear current trajectory queue and replace with a new set).

   (c) `recvfbk` - Triggered by feedback from `/joint_states`, this function updates the internal state of the Lowlevel Node with the robot's current joint positions, velocities, and torques.

   (d) `update` - This is a timer-based function that runs at 100 Hz. It continuously evaluates the active spline, computes interpolated joint positions and velocities, and publishes them to `/joint_commands`. It also checks for collisions by comparing measured joint torques against expected values and, if necessary, publishes an `ExecutionStatus` message indicating a collision.

5.2 Custom messages

For `detected_objects`, `user_ball`, `spline_segments`, `control_commands` and `execution_status`, we defined 5 different kinds of custom messages.

- `Object.msg`: This is used to represent any object detected on the table, such as the golf ball or the club. It contains:
  - `int32 type` – Specifies the type of object, e.g., `DISK` = 0 for golf ball and `STRIP` = 1 for the golf club.
  - `geometry_msgs/Pose pose` – Contains the position (x, y, z) and orientation (as a quaternion) of the detected object in the world frame.
- `ObjectArray.msg`: This is used by the `golf_detector` to publish all detected objects in a single message. It contains:
  - `Object[] objects` – An array of `Object.msg` messages representing all detected objects (e.g., the ball and the club).
- `SplineSegment.msg`: This is used to represent a single trajectory (spline) that the lowlevel node should execute. It contains:
  - `float64[] qf` – The target joint positions (goal of the spline).
  - `float64[] vf` – The target joint velocities (final velocities after executing the spline).
  - `float64 t_move` – The time the lowlevel node should take to execute the spline.

- `ControlCommand.msg`: This is used for sending control signals from the brain to the lowlevel node when special behavior is required (e.g., emergency stop or trajectory replacement). It contains:
  - `string command` – A string that is either `"halt"` or `"replace"`.
  - `SplineSegment[] new_segments` – A list of new spline segments used only when the command is `"replace"`.
- `ExecutionStatus.msg`: This is used by the lowlevel node to report its status back to the brain. It contains:
  - `bool ready` – Indicates whether the lowlevel node has completed its spline queue and is ready for new commands.
  - `bool collision` – Indicates whether a collision has been detected based on torque measurements.

6 Modeling/Understanding the World.

The Brain node serves as the central decision-making component of our mini-golf robot system. It is responsible for collecting and processing sensor data, managing world state, and generating high-level commands. This node builds a reactive and efficient state machine, where the `self.ready` flag acts as a gatekeeper to control when decisions can be made.

6.1 Data inputs overview

The Brain node receives two major streams of data:

- **From `golf_detector`:**
  - `/golfdetector/detected_objects`: Publishes `ObjectArray` containing detected objects such as the robot's ball (type `DISK`), the club (type `STRIP`), and other objects.
  - `/golfdetector/user_ball`: Publishes `Object` containing the user's ball position.
- **From `lowlevel`:**
  - `/execution_status`: Publishes `ExecutionStatus`, signaling when the robot is ready for the next command (`ready = True`) or when a collision has occurred (`collision = True`).
  - `/joint_states`: Publishes `JointState` messages with feedback on joint positions, velocities, and torques.

6.2 Central control mechanism: `self.ready`

The Brain only processes sensor data from `/detected_objects` when `self.ready = True`. Once a spline trajectory is issued to the lowlevel, `self.ready` is immediately set to `False`, locking further decision-making. Only upon receiving `execution_status.ready = True` does the Brain return to the decision state (`self.ready = True` again), if a collision is not detected.

6.3 Tracked internal states

The Brain maintains the following internal states to make decisions:

- **Ball position** (robot ball): `ball_position`
- **Club status**: `self.club_picked_up` (whether the club has been gripped)
- **Ball starting position status**: `self.ball_moved_to_start`
- **Current turn**: `self.robot_turn = True` (robot's turn) or `False` (user's turn)
- **Collision status**: from `execution_status`
- **User ball position**: `self.user_ball_position`
- `self.ready` **flag**: Whether the robot is free to act.

6.4 State flow overview

6.4.1 Initialization

On startup, the robot initializes its position using `self.grabfbk()` and sets `self.ready = True` to allow the first decision to happen.

6.4.2 User turn monitoring (Passive mode)

Condition: `self.robot_turn = False`

The robot listens to `/user_ball` messages via `recv_user_ball()`. A hybrid detection method accumulates ball movement over time: once cumulative movement exceeds a threshold (ball was hit) and then stops for a continuous time window (e.g., 0.8s), the Brain:

- Switches to `self.robot_turn = True`
- Sets `self.ready = True` to begin robot actions.

6.4.3 Robot decision state (`self.ready = True` and `self.robot_turn = True`)

Inside `recv_objects()`, the Brain analyzes current object positions and internal states to select one action from the following, prioritized as below:

A. Search behavior:

- **Club Search:** If the club is not detected and has not yet been searched for (`self.has_moved_club = False`):
  - The robot executes a small base rotation to "look around".
  - Publishes this search movement as a spline.
  - Sets `self.ready = False` to wait for the search motion to complete.
- **Ball Search:** If the ball is not detected and search attempts (`self.has_moved_ball`) are less than 3:
  - Same search behavior occurs.
  - `self.has_moved_ball` is incremented.

B. Move ball to starting position: If the ball is detected but has not yet been moved to its starting location (`self.ball_moved_to_start = False`):

- The Brain computes a safe path and physically moves the ball to its start point.
- Once placed, `self.ball_moved_to_start = True`.
- `self.ready = False` after commanding the move.

C. Wall avoidance (Ball by wall check): If the ball is near a wall (checked via `by_wall()` function), the Brain:

- Plans a corrective move to reposition the ball centrally.
- Drops the club first if it is currently held.
- Executes the ball repositioning move.

D. Club pickup: If the club is detected and `self.club_picked_up = False`:

- Executes the pickup routine (`pickup_club()`), including IK-based approach, gripper closure, and lifting.
- Marks `self.club_picked_up = True` afterward.

E. Swinging or putting: If the ball is stationary (`is_ball_stopped()` returns True) and the club is ready:

- The Brain checks if the ball is in a "swingable" area (`swingable()` function):
  - If **swingable**, it performs a full swing (`hit_ball()`).
  - If **not swingable**, it executes a soft putt (`putt()`).
- After the hit/putt:
  - `self.robot_turn = False` (giving turn back to user).
  - `self.ready = False` while the swing/putt finishes.

6.4.4 Robote execution / waiting state (`self.ready = False`)

The robot ignores new `/detected_objects` messages but continues receiving `/execution_status`. It passively waits for one of two triggers:

- `execution_status.ready = True`: Indicates splines are complete. Sets `self.ready = True` to resume planning.
- `execution_status.collision = True`: Immediately triggers `send_halt_command()` and asks the robot to come to a complete stop (no position or velocity or torque being sent to motors). Initally, we used to send a command to replace the current trajectory with a trajectory to go home, but then we realized there would be people walking around and if collision was detected, it would be best to stop moving instead of "move more" to go home.

6.5 Important internal logic details

Turn handling: `self.robot_turn = True` enables `/detected_objects` processing. After the robot hits or putts, `self.robot_turn = False` to wait for user action.

Search logic details: `self.move_robot_slightly()` moves the base joint to "scan" the area for missing objects. Only triggers once per search cycle using:

- `self.has_moved_club` for the club
- `self.has_moved_ball` (limited to 3) for the ball

Object manipulation flow: All pickup/drop/putt/hit actions follow this pattern:

1. Compute pose with IK (e.g., `nr_ikin()`, `xyt_ikin()`).
2. Move above object.
3. Lower to object.
4. Gripper open/close.
5. Raise and reposition as needed.
6. Set `self.ready = False` immediately after publishing the spline.

6.6 Why `self.ready` is essential

Without this lock:

- Multiple `/detected_objects` frames could be processed within milliseconds, resulting in:
  - Conflicting spline commands sent to the lowlevel node.
  - Misaligned ball/club states due to mid-motion occlusions.
- The robot could replan while still executing a trajectory, causing feedback-control instability.

`self.ready` ensures:

- Planning happens only when fully stationary.
- Splines are executed to completion before reprocessing the scene.
- Robot acts on fresh and valid sensor data, never on stale frames.

7 Behaviors and Failure Recoveries

7.1 Nominal Game Sequence

The system exhibits the following core behaviors in a typical game loop:

- **Wait for Opponent's Turn**
  The robot passively monitors the user's ball via the `/golfdetector/user_ball` topic.
  Once the system detects that the user's ball has been hit and stopped, the robot switches to active mode (`self.robot_turn = True`).

- **Move Ball to Starting Position**
  On the first turn, if the robot ball is not at its predefined starting location, the robot grips the ball and moves it to the start.
  This ensures a consistent opening position for subsequent gameplay.

- **Take the First Swing**
  The robot picks up the club (if not already holding it).
  Evaluates if the ball is in a "swingable" or "puttable" position.
  Executes a full swing or soft putt depending on ball location.

- **Alternate Turns**
  After executing its shot, the robot waits for the opponent's next turn.
  This cycle repeats until one of the players successfully sinks their ball into the hole.

7.2 Swingable vs. Puttable Zones

The system divides the playable field into two distinct types of zones:

- **Swingable Zones**: Areas where there is sufficient space around the ball for the robot to perform a full backswing and follow-through. Typically located away from walls and field boundaries.

  Behavior: The robot positions itself above the desired contact position with the ball, and swings downwards - like a full golf swing.

- **Puttable Zones**: Areas near the walls or within tight spaces where a full swing is not mechanically feasible or safe.

  Behavior:
  The robot positions itself straight right behind the ball and swings slightly forward - pushing the ball toward the desired direction. The Brain decides which behavior to trigger using the `swingable()` function, which evaluates ball position relative to walls, obstacles, and predefined swing-clearance boundaries.

7.3 Recovery Modes

To handle common failure modes and edge cases, the system implements robust recovery behaviors.

- **Wall proximity recovery (Ball near wall)**
  - **Trigger condition:** If `by_wall()` detects the ball is too close to a wall (e.g., within 10-15 cm), where neither a swing nor a putt would be effective.
  - **Recovery action:** The robot drops the club at a safe intermediate location, grips the ball, and moves it approximately one club's length away from the wall into a central, playable zone. It then re-picks the club once the ball is relocated.

- **Club pickup retry**
  - **Trigger condition:** After attempting to pick up the club, the system checks if the gripper's effort readings confirm a successful grip. If not, and `self.tried_picking_club < max_club_pick_attempts` (usually 2 retries allowed), it will try again.

- **Recovery action:** The robot re-attempts the pickup from the current position or slightly adjusts its position to improve grip success. If retries are exhausted, the robot continues gameplay assuming the club is held, as a fallback.

- **Ball not hit recovery**
  - **Trigger condition:** If the system detects that after a swing or putt, the ball did not move (or moved below a small distance threshold), it assumes the hit was unsuccessful.
  - **Recovery action:** The robot immediately reattempts the swing or putt and recalculates the motion based on the updated ball position.

- **Ball or club covered?**
  - **Trigger condition:** When either the ball or the club cannot be detected in `/detected_objects` for several frames in a row.
  - **Recovery action:** The robot executes a small adjustment (moves the base motor by $\frac{\pi}{4}$ using `move_robot_slightly()`) to move the robot out of the way and re-detect the missing object(s). This is limited to a fixed number of attempts (e.g., 3 tries) before stopping further retries. If the ball is still not detected, the robot assumes the ball has gone into the hole.

- **Collision detection recovery**
  - **Trigger condition:** The `/execution_status` topic reports `collision = True` during spline execution.
  - **Recovery action:** The robot immediately sends a halt command to the lowlevel node to stop all motion. Optionally, it may replace the trajectory with a pre-defined safe "go-home" spline to reset the arm to its neutral position. After halting, the robot can retry the action or abort depending on the scenario.

8 Shortcomings, Outstanding Items, and Lessons Learned

8.1 Error mapping

As mentioned previously, we realized that our error map should account for multiple branches for every (x, y) position on the course. However, due to the lack of time, we were unable to implement this error map and opted for one that mapped (x,y) to (x, y, z) error. Though this map worked for certain locations on the course, it proved inadequate for key areas and heavily impacted performance and gameplay. Ultimately, opting for joint-coordinate-based error mapping would be ideal.

8.2 Mechanical design

Improving the rigidity of the arm proved difficult throughout the project, and despite making significant improvements to the initial interaction, the arm still exhibited some sagging, introducing great amounts of error to the system. Though the removal of the "spoon" motor mounts improved joint stiffness, further work can be done to make the joints stiffer, particularly by redesigning the "fork" motor mounts. Our current "fork" mounts concentrate a significant portion of the load on a small cross-sectional area, therefore, redesigning these mounts to have a larger cross-sectional area would improve joint stiffness.

As noted previously, we encountered several issues gripping objects with accuracy and precision. To address these issues, along with several gripper design iterations, we started working on the implementation of haptic grip detection for the club. Unfortunately, due to time constraints, we were unable to fully test this code. Fully integrating this code into our recovery modes not just for the club but for the ball could have made significant improvements to our accuracy and precision.

Additionally, cable management could have been improved so that the cables do not extend too far out from the links. Due to our required range of motion, the cables needed to be slack enough so link movements weren't restricted but taught enough so the cables would not get caught on anything as the arm moved around. However, occasionally, the cables would block parts of the club and/or the ball, which would lead to detection errors, causing problems with positioning and orientation. Better cable management strategies could have been employed, such as running our cables inside of the aluminum tubing rather than attaching them externally, to help mitigate these issues.

8.3 Human interaction

The final version of brain pipeline included a rudimentary turn-taking process. While it worked at times, it seemed to skip the other player's turn, especially after putting. It would say that the other player took their turn when they hadn't yet. We did not have enough time to debug whether this was a detector sensitivity issue, a logic issue, or some other issue. A further developed version of the human interaction pipeline could store the number of turns each player took, handle more than one opponent, remember the positions of each ball in case they were moved, move out of the way when not its turn, and detect when humans are reaching into the course area.

8.4 Outstanding Items

In addition to reducing sag, an additional camera could have been mounted to the gripper to improve gripping accuracy. By providing a closer, more direct view of the end effector's position, errors caused by distortion and disruptions to the field of view from the overhead camera would be reduced. Furthermore, this secondary camera would allow for finer adjustments during object manipulation improving precision in positioning and orientation.

An important aspect of this project was demonstrating our robot's ability to recover from unexpected behavior. Consequently, we deemed it best not to implement any planning algorithms that could consistently achieve hole-in-ones. This decision allowed us to demonstrate our recovery modes and the game to remain engaging, avoiding the need for additional obstacles to maintain its challenge. However, introducing planning algorithms to this system would be an interesting area to explore.

Furthermore, the robot can only swing at a set speed regardless of the distance it is from the desired position. However, this is unideal and often results in the ball landing in an unswingable or unputtable area or overshooting the hole when close. To improve performance, the swing speed should be adjustable based on distance, an improvement that we could not implement due to time constraints.

8.5 RL-based joint-space trajectories to avoid collisions

One of the main challenges in our robotic system was avoiding collisions with static obstacles, namely the mini-golf course walls and the robot's mounting base while reaching the ball from varying positions. Given that the obstacles were stationary, we explored reinforcement learning (RL) as a method to learn joint-space trajectories that inherently avoid these obstacles while moving

toward the ball. We aimed to develop a policy that generalizes across different ball positions while complying with the robot's kinematic and environmental constraints.

Since the only variable in the environment was the ball position, we hypothesized that RL could discover smooth, continuous joint movements that balance efficiency in reaching the ball with the avoidance of collisions.

We modeled this as a continuous control problem where the RL agent outputs joint angle deltas $[\Delta q_1, \Delta q_2, \Delta q_3, \Delta q_4, \Delta q_5]$ at each time step. These deltas are incremental updates to the robot's five degrees of freedom. The agent's state includes both the current joint angles $[q_1, q_2, q_3, q_4, q_5]$ and the ball position $[x_{\text{ball}}, y_{\text{ball}}]$ in the environment frame.

The reward function was designed to encourage the robot to approach the ball while avoiding collisions. It strongly penalizes collisions, rewards proximity to the ball, and discourages the robot from getting too close to the walls or mounting base, even without a direct collision. This was achieved using an exponentially shaped reward:

$$R = \begin{cases} -100, & \text{if collision occurs} \\ +50 \cdot e^{-5 \cdot d_{\text{ball}}}, & \text{for approaching the ball} \\ -5 \cdot e^{-10 \cdot d_{\text{wall}}}, & \text{if near a wall} \\ -5 \cdot e^{-10 \cdot d_{\text{mount}}}, & \text{if near the mounting base} \end{cases}$$

where d_{ball} is the distance between the end-effector and the ball, d_{wall} is the minimum distance to the walls, and d_{mount} is the distance to the mounting clearance zone. The exponential terms encourage smooth trajectories while softening penalties for near misses, except in the case of hard collisions.

To conduct training, we created a custom ROS2 simulation node that models the robot's interaction with its environment. This node receives the joint deltas from the RL agent, updates the robot's internal joint states, computes forward kinematics using our `KinematicChain.fkin()` function to locate the end-effector, performs collision checks, calculates the reward, and publishes the updated observation back to the RL agent.

The ROS interfaces for this environment included the `/rl_agent_action` topic to receive joint deltas, `/rl_observation` to publish the updated state, reward, and done flag, and a `/reset_env` service to reset joint angles and ball positions for new episodes.

The control loop involved receiving a joint delta, updating joint angles, computing forward kinematics to determine the end-effector position, performing collision checks, computing the reward, and finally publishing the updated observation to the agent.

The RL agent node interfaced directly with the ROS environment. It was responsible for learning the joint delta policy, publishing actions, processing observations and rewards, and managing environment resets. For the learning algorithm, we selected Soft Actor-Critic (SAC) just as an exploratory attempt.

We decided against this approach as it was computationally excessive for our scenario, especially since our Newton-Raphson method was working better with far less overhead. Incorporating rotation matrices, which we had not yet done, would have increased the complexity and required more time for testing and learning. Also, it was unclear whether the RL policy would reliably generalize to all ball positions. Given all this, we prioritized getting our existing NR to work. But, it was really cool to see how RL could be applied to robotics, and we think it could work for more complex environments or robotic systems.

8.5.1 LLM controlling the brain

We experimented with using the GPT-4o API as a replacement for our brain node. We gave the LLM all detector outputs, background information about the robot's objectives, and the current state variables, allowing it to decide the next action. However, the LLM didn't know the game flow logic to make good decisions. Approximately 40% of the time, it generated incorrect next steps—for instance, choosing to hit the ball directly without first checking its proximity to walls.

To improve accuracy, we added more explicit logic and task sequencing into the prompt, which slightly improved the results. But still, the LLM's inference time was around 5 seconds per decision, which is much slower than our existing brain implementation, which completes all processing—including Newton-Raphson convergence—in under a second.

Given the latency and complexity, we realized this approach was overkill and opted to retain our state-machine-based brain. Had we more time, we would have explored triggering specific robot functions based on the LLM's outputs and optimizing coordination between the low-level and brain nodes. But, we didn't think this was possible as the brain's decisions depend on the completion of spline executions by the low-level node, limiting overlap in their execution timelines.